



Тестирање на апликација - EduMeet

Ментори:

Проф. Др. Христина Михајлоска

Асс. Ана Тодоровска

Изработил:

Димитар Арсов 231017

Линк до репозиториум: <https://github.com/Dimitar-04/edumeeet>

Документација за тестирање на EduMeet

Овој документ ја опишува имплементацијата и резултатите од тестирањето на EduMeet. Тестирањето опфаќа формална стратегија за партиционирање на влезниот простор, мутациско тестирање, unit, компонентни, интеграциски, API, end to end и перформансни тестови.

Содржина

Поглавје	Содржина
1	Вовед
2	Формална стратегија ISP со Base Choice Coverage
3	Мутациско тестирање со Stryker.NET
4	Backend unit тестови со xUnit
5	PostgreSQL интеграциски тестови
6	Frontend компонентни тестови
7	HTTP API тестови
8	End-to-end тестови со Playwright
9	Перформансно тестирање со k6
10	Автоматизација на извршувањето
11	Резултати и оценка на тестирањето
12	Заклучок

1 Вовед

EduMeet е full stack платформа за пронаоѓање, организирање и посетување образовни настани. Индивидуалните корисници можат да се регистрираат за настан, да добијат код за присуство, да бидат евидентирани преку QR скенирање и по завршувањето да остават рецензија. Корисниците (индивидуи и организации) креираат настани, управуваат со присуството и ги следат пријавените учесници.

1.1 Цел и опфат

Тестовите се избрани според ризикот. Најдетално се проверуваат операциите што менуваат трајна состојба, зависат од авторизација, време или надворешен сервис и минуваат низ повеќе слоеви. Тривијални getters, setters и статички елементи на интерфејсот не се тестираат посебно.

Ниво или техника	Главна цел	Алатки
ISP со BCC	Систематски избор на влезни блокови за критичен метод	xUnit и NSubstitute
Мутациско тестирање	Проверка дали тестовите откриваат намерни промени во логиката	Stryker.NET
Backend unit тестови	Изолирана проверка на деловни и инфраструктурни правила	xUnit и NSubstitute
Frontend компонентни тестови	Однесување на компоненти и API клиентот	Vitest, React Testing Library, MSW
Интеграциски тестови	EF Core, миграции и PostgreSQL правила	Testcontainers
HTTP API тестови	Целосен ASP.NET Core pipeline и HTTP договори	WebApplicationFactory и Testcontainers
End to end тестови	Реални кориснички текови преку browser	Playwright и Docker Compose
Перформансни тестови	Латентност и стабилност под паралелно оптоварување	k6 и Docker Compose
Автоматизација	Повторливо локално и CI извршување	PowerShell и GitHub Actions

2 Формална стратегија ISP со Base Choice Coverage

2.1 Избор на метод

Формалната стратегија е применета само на методот `EducationalEventService.CheckInParticipantAsync`. Методот е избран затоа што е деловно критичен и содржи повеќе независни одлуки: постоење на организатор, постоење и сопственост на настан, временски прозорец, валидност на attendance token и претходна состојба на присуството. Грешка во оваа логика може да дозволи неовластено или погрешно евидентирање на учесник.

Користено е функционално партиционирање. Карактеристиките се изведени од деловното однесување и состојбата на репозиториумите, а не само од типовите во потписот на методот. Base Choice Coverage избира еден најчест валиден блок како база, а потоа во секој дополнителен тест менува еден блок додека останатите применливи блокови остануваат базни.

2.2 Карактеристики и блокови

Карактеристика	Базичен блок	Спореден блок
Организатор	Постои	Не постои
Настан	Постои	Не постои
Сопственост	Организаторот е сопственик	Организаторот не е сопственик
Време на check in	Прозорецот е отворен	Пред отворање или по затворање
Attendance token	Одговара на учесник	Не одговара
Состојба на присуство	Не е претходно евидентиран	Веќе е евидентиран

Базниот случај содржи постоечки организатор, постоечки настан во сопственост на организаторот, отворен временски прозорец, точен token и учесник кој не бил претходно евидентиран. Очекуваниот исход е успешно прво евидентирање и едно зачувување преку Unit of Work.

2.3 Изведени BCC тестови

Случај	Променет блок	Очекувано однесување
BC1	Нема промена	Успешен check in и едно зачувување
BC2	Организаторот не постои	NotFoundException и прекин пред читање на настанот
BC3	Настанот не постои	NotFoundException без hashing и зачувување
BC4	Погрешен организатор	ForbiddenException без промена на состојба
BC5	Пред отворање	ConflictException
BC6	По затворање	ConflictException
BC7	Невалиден token	NotFoundException без зачувување
BC8	Веќе е евидентиран	Се враќа постојната состојба без повторно зачувување

Тестовите не проверуваат само исклучок или повратна вредност. Со NSubstitute се проверува и дали зависностите по неуспешниот guard воопшто не биле повикани. DidNotReceive за HashToken и SaveChangesAsync докажува дека методот прекинал на точната гранка и дека не оставил несакана промена.

3 Мутациско тестирање со Stryker.NET

Мутациското тестирање е применето на истиот метод за да се оцени силата на BCC тестовите. Stryker создава изменети верзии на production кодот, на пример замена на споредбен оператор, негација на услов или отстранување на израз. Потоа ги извршува избраните тестови.

```
"test-case-filter":  
"FullyQualifiedName~Application.UnitTests.Services.EducationalEventServiceCheckInTest"
```

```
s"
"mutate": ["**/EducationalEventService.cs{16365..19384}"]
```

Филтерот за тестови го ограничува извршувањето на класата `EducationalEventServiceCheckInTests`, а `mutate` опсегот го ограничува менувањето на `source span` што го содржи `CheckInParticipantAsync`. На овој начин резултатот се однесува на избраниот метод, а не на целата `service` класа.

Првата анализа создаде 28 релевантни мутанти. ВСС тестовите убија 26, додека преживеаја две промени на граничните временски споредби. Промената `nowUtc < opensAt` во `nowUtc <= opensAt` и промената `nowUtc > closesAt` во `nowUtc >= closesAt` не беа разликувани од почетните случаи. Затоа беа додадени два теста: `check in` точно во моментот на отворање и точно во моментот на затворање. Двата гранични моменти се валидни според `production` логиката.

Фаза	Убиени	Преживеани	Резултат
Почетен ВСС сет	26	2	Потребни се гранични тестови
По додавање на границите	28	0	100 проценти за избраниот опсег

Овој резултат не значи дека целата апликација има 100 проценти мутациска покриеност. Резултатот важи само за намерно ограничениот метод и избраната група тестови.

4 Backend unit тестови

Backend unit тестовите се имплементирани со `xUnit`. `NSubstitute` ги заменува репозиториумите, `Unit of Work`, `file storage`, `attendance token service` и `email outbox`. На овој начин секој тест го проверува однесувањето на една класа без реална база, датотечен систем или `SMTP` сервер. За логиката зависна од време се користи `FixedTimeProvider`, со што резултатите не зависат од моментот на извршување.

Тест проект	Број на тестови	Проверена функционалност
Application UnitTests	19	Check in, регистрација за настан и рецензии
Infrastructure UnitTests	8	JWT и refresh token, attendance token
Presentation UnitTests	3	Читање и валидација на прикачени слики
Вкупно	30	Изолирани backend правила

4.1 Application service тестови

Покрај десетте `check in` тестови, регистрацијата за настан проверува успешна пријава со генерирање `token` и `outbox` порака, повторна операција што го одјавува корисникот и одбивање на организациска сметка, сопствен настан или веќе започнат настан. Тестовите за рецензии проверуваат успешна рецензија од присутен учесник, забрана за неприсутен учесник, `self review` од организатор и дупликат рецензија.

За успешните случаи се проверуваат создадените `domain` објекти, пресметаните броеви и повиците кон `SaveChangesAsync`. За одбиените случаи се проверува типот на исклучок и отсуството на трајна промена.

4.2 Infrastructure и presentation тестови

AttendanceTokenService се проверува за формат XXXX-XXXX-XXXX, нормализација на различно внесени кодови и детерминистичко hashing однесување. TokenService тестовите ги читаат создадените JWT claims и го проверуваат времето на истекување. Refresh тестовите опфаќаат валиден, истечен и повлечен refresh token.

FormFileReader е тестиран на границата со ASP.NET Core IFormFile. Поддржана слика се претвора во податочен објект, додека преголема датотека и неподдржан MIME тип се одбиваат пред да стигнат до file storage.

5 PostgreSQL интеграциски тестови

Интеграциските тестови користат Testcontainers PostgreSQL и реална PostgreSQL 17 инстанца. Ова е избрано наместо in memory provider бидејќи SQL ограничувањата, case insensitive пребарувањето, сортирањето и однесувањето на миграциите можат да се разликуваат од production provider.

PostgreSQLFixture го имплементира IAsyncLifetime. InitializeAsync го стартува контејнерот, го создава ApplicationDbContext со динамичката connection string и ги применува сите EF Core миграции. Пред секој тест ResetDatabaseAsync ги празни application табелите со TRUNCATE RESTART IDENTITY CASCADE, но ја задржува табелата за историја на миграции. DisposeAsync го отстранува контејнерот по колекцијата тестови.

Бр	Интеграциска проверка
1	Сите EF Core миграции се применуваат
2	Не може да постои дупликат регистрација за ист настан
3	Attendance token е единствен кога има вредност
4	Настан мора да има постоечки организатор
5	Пребарување без разлика на големи букви, филтрирање, редослед и pagination
6	Распоредот содржи само пријавени идни настани
7	Историјата бара check in и го почитува category филтерот
8	Email outbox состојбата се зачувува низ enqueue, retry и processing

6 Frontend компонентни тестови

Frontend тестовите се извршуваат со Vitest во jsdom околина. React Testing Library ги наоѓа контролите преку улоги и labels, а user event симулира внесување текст, кликови и upload на датотека. Тестовите го проверуваат однесувањето што го гледа корисникот, без да зависат од внатрешната структура на React компонентите.

MSW ја пресретнува HTTP комуникацијата на ниво на request boundary. Во заедничкиот setup, непознат мрежен повик се третира како грешка, а handlers и mocks се ресетираат по секој тест. Google Maps и camera scanner се mock-ираат бидејќи нивното реално однесување припаѓа на надворешни сервиси и физички уреди.

Група	Број на тестови	Главни проверки
Axios client	4	Успешно барање, refresh и retry, еден refresh за конкурентни 401, неуспешен refresh
Auth API и context	3	Multipart регистрација, restore на сесија, logout и refresh failure
SignUp	4	Валидна организација и image, passwords, големина на image, backend error
LocationPicker	3	Почетни координати, избор на place, безбедна грешка при initialization
AttendanceQrScanner	2	Еднократно праќање на scan и camera permission error
Вкупно	16	Компоненти и frontend интеграциски граници

7 HTTP API тестови

API тестовите повикуваат реални HTTP endpoints преку Microsoft ASP.NET Core WebApplicationFactory. За разлика од controller unit тест, тука се активни routing, model binding, validation, authentication, exception handler, application services, repositories, Entity Framework Core и PostgreSQL.

ApiTestFixture стартува посебен PostgreSQL Testcontainer, ги применува миграциите и создава EduMeetApiFactory. Фабриката го поставува Testing environment и ги обезбедува connection string, JWT и email поставките пред стартување на Program.cs. Production DbContext registration се заменува со базата од контејнерот, background email worker се отстранува, времето се фиксира, а physical file upload се заменува со TestFileUploadService.

Област	Број на тестови	Опфат
Автентикација	4	Регистрација и session cookie, дупликат, login refresh logout, погрешна лозинка
Настани	3	Креирање и читање, pagination validation, анонимен create
Учество и присуство	5	Toggle регистрација, тип на сметка, валиден и невалиден check in, погрешен организатор
Профил	3	Јавен профил, username и image update, анонимен update
Рецензии	3	Create и delete, неприсутен учесник, grade validation
Вкупно	18	HTTP статуси, тела, cookies и database state

8 End to end тестови со Playwright

Playwright ги тестира најважните текови преку Chromium и реалниот frontend. Тестовите не ги повикуваат React компонентите директно, туку отвораат страници, пополнуваат форми и кликаат контроли како корисник. Frontend, backend, authentication и PostgreSQL остануваат реални.

Бр	Корисничко сценарио
1	Гостин прегледува, пребарува, филтрира и отвора детали за настан
2	Индивидуален корисник се регистрира, се најавува, ја обновува сесијата и се одјавува
3	Корисник ажурира профил и ја гледа зачуваната промена
4	Организација креира настан и го гледа во апликацијата
5	Индивидуален корисник се пријавува за настан и го гледа во распоредот
6	Организатор евидентира учесник, а учесникот создава и брише рецензија

9 Перформансно тестирање со k6

Перформансниот тест ги таргетира јавните read операции: листа на претстојни настани, пребарување со category филтер и детали за конкретен настан. Избрани се read операции затоа што претставуваат најчеста интеракција и овозможуваат повторлив товар без создавање нови записи во секоја итерација.

Функцијата setup се извршува еднаш. Таа чека backend да биде достапен, преку реалното API регистрира уникатна организација и создава иден настан. Вратените eventId, title и searchTerm се споделуваат со виртуелните корисници. Овие setup барања се означени посебно и не влијаат врз measured thresholds.

Warmup сценариото постепено додава до три виртуелни корисници во 10 секунди. Потоа constant vus сценариото одржува 10 паралелни корисници во 60 секунди. Секој корисник последователно повикува list, search и details, ги проверува статусот и содржината, чека една секунда и ја повторува итерацијата.

Критериум	Праг	Потврден резултат
Неуспешни measured HTTP барања	Помалку од 1 процент	0 проценти
Measured p95 response time	Под 750 ms	31,95 ms
Успешни checks	Над 99 проценти	100 проценти
Обем на measured фазата	10 VUs во 60 секунди	1 737 HTTP барања

Во потврдениот run биле завршени 589 итерации и 1 770 HTTP барања вкупно, од кои 1 737 припаѓаат на measured фазата. Овие вредности се baseline за конкретната машина и Docker околина, а не гаранција за production капацитет.

10 Автоматизација на извршувањето

10.1 Унифицирана локална скрипта

run-tests.ps1 претставува единствена локална влезна точка. Параметарот Suite прифаќа BackendUnit, Frontend, Integration, Api, E2E, Mutation, Performance или All. Invoke-TestStep го печати името, ја извршува командата и го проверува \$LASTEXITCODE. Скриптата прекинува на првиот неуспешен suite. Invoke-InDirectory се користи само за npm, Playwright и Stryker, каде што тековниот директориум е потребен за package.json или config датотеката.

```
powershell -ExecutionPolicy Bypass -File .\run-tests.ps1
```

Performance параметрите овозможуваат промена на бројот на VUs и времетраењето без промена на k6 датотеката. Посебната performance/run.ps1 скрипта ја стартува Compose околината, го зачувува k6 exit code и секогаш ја чисти околината во finally блок.

10.2 GitHub Actions

CI workflow се активира при push, pull request и рачно преку workflow dispatch. Backend, frontend, PostgreSQL integration и HTTP API jobs се извршуваат како одделни проверки. Chromium E2E job започнува само ако претходните jobs поминале. На овој начин неуспехот е јасно поврзан со слојот во кој настанал, а поскапиот системски тест не се стартува кога основните проверки веќе паднале.

11 Резултати и оценка на тестирањето

Група	Резултат
Backend unit тестови	30 успешно поминати
PostgreSQL интеграциски тестови	8 успешно поминати
HTTP API тестови	18 успешно поминати
Frontend компонентни тестови	16 успешно поминати
Playwright E2E сценарија	6 успешно поминати
Мутациско тестирање	28 од 28 мутанти убиени во избраниот метод
k6 measured load	0 проценти failures и p95 31,95 ms
GitHub Actions	Сите jobs успешно поминати

Комбинацијата од тестови дава различни видови докази. Unit тестовите брзо ги локализираат грешките во правилата, интеграциските тестови го проверуваат вистинското database однесување, API тестовите го проверуваат серверскиот pipeline, а Playwright докажува дека главните текови работат преку реалниот интерфејс. ISP обезбедува систематски избор на случаи за критичниот метод, додека Stryker покажа дека граничните грешки навистина се откриваат.

12 Заклучок

Спроведеното тестирање дава основана доверба дека критичните функционалности на EduMeet се однесуваат правилно во очекуваните, граничните и неуспешните сценарија. Комбинирањето различни нивоа и техники овозможи грешките да се разгледаат од повеќе перспективи, наместо квалитетот да се проценува само според бројот на тестови или процентот на покриеност.